
Universidad Técnica Estatal de Quevedo

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL

CARRERA DE INGENIERÍA EN SOFTWARE

Informe Técnico Final

Administración de Bases de Datos

Proyecto SBVIA

Simulador de Comportamiento Vial con Inteligencia Artificial

Asignatura: Administración de Bases de Datos

Nivel / Paralelo: 5to Nivel — Paralelo B

Docente: Ing. Cordero Bazurto José Steven, SOFT-R

Integrantes: Cruz Pérez Justyn Keith
Umaginga Arévalo Jefferson Manuel
Zamora Bumbila Diego Alexander

Motor de BD: PostgreSQL 16

Repositorio: github.com/keithdrox/SBVIA-AppWeb

Índice general

1. Introducción	3
1.1. Descripción del problema	3
1.2. Objetivo del sistema	3
1.3. Alcance del informe	4
2. Arquitectura General del Sistema	5
2.1. Visión general	5
2.2. Módulos principales del backend	5
2.3. Flujo de una simulación	6
2.4. Relación con la base de datos	6
3. Diseño de la Base de Datos y Millón de Registros	7
3.1. Descripción del modelo relacional	7
3.2. Estructura de tablas principales	7
3.2.1. Tabla <code>usuario</code>	7
3.2.2. Tabla <code>simulacion</code>	8
3.3. Relaciones e integridad referencial	8
3.4. Restricciones CHECK implementadas	8
3.5. Triggers de automatización	9
3.6. Vistas del sistema	10
3.7. Generación del millón de registros	10
3.7.1. Estrategia de carga masiva	10
3.7.2. Distribución de registros generados	11
3.7.3. Consulta de verificación	12
4. Implementación y Gestión de Respaldos	13

4.1. Herramienta: <code>pg_dump</code>	13
4.2. Generación del respaldo	13
4.3. Respaldo desde contenedor Docker	13
4.4. Restauración	14
4.5. Script de respaldo automatizado	14
5. Auditoría de Operaciones de la Base de Datos	16
5.1. Sistema de auditoría de dos niveles	16
5.1.1. Nivel 1: Auditoría de accesos	16
5.1.2. Nivel 2: Trigger de auditoría automática	16
5.2. Consultas de auditoría	17
5.3. Auditoría DML extendida	18
6. Control de Usuarios, Roles y Permisos	19
6.1. Roles del sistema (nivel de aplicación)	19
6.2. Usuarios PostgreSQL con permisos diferenciados	19
6.3. Demostración de restricción de acceso	20
6.4. Vista segura para columnas sensibles	21
7. Índices y Optimización de Consultas	22
7.1. Índices implementados	22
7.2. Creación de índices	23
7.3. Análisis EXPLAIN ANALYZE	24
7.3.1. Consulta 1: Historial de simulaciones de un usuario	24
7.3.2. Consulta 2: Auditoría de accesos recientes	25
7.4. Glosario de campos EXPLAIN ANALYZE	25
7.5. Verificación del uso de índices	25
8. Conclusiones	27
Referencias	28

Capítulo 1

Introducción

1.1. Descripción del problema

Los siniestros viales representan una de las principales causas de mortalidad mundial, especialmente entre conductores novatos que carecen de suficiente exposición práctica a situaciones de riesgo. La formación tradicional limita la cantidad y diversidad de escenarios que un conductor puede enfrentar antes de obtener su licencia, lo que se traduce en deficiencias en la toma de decisiones bajo presión, tráfico denso o condiciones climáticas adversas.

El proyecto **SBVIA** (*Simulador de Comportamiento Vial con Inteligencia Artificial*) surge como respuesta a esta problemática: una plataforma web que permite a conductores en formación practicar decisiones viales en escenarios virtuales controlados, registrando cada acción, infracción y métrica de desempeño. El sistema proporciona retroalimentación automática y evaluaciones por inteligencia artificial, complementando la formación práctica supervisada sin reemplazarla.

1.2. Objetivo del sistema

Desarrollar y validar una plataforma web reproducible que permita:

- Registrar y evaluar el comportamiento vial de conductores en escenarios virtuales con distintos niveles de dificultad.
- Generar retroalimentación automática basada en infracciones detectadas y métricas de desempeño.
- Proveer a instructores y administradores de reportes detallados sobre el progreso de los participantes.
- Mantener trazabilidad completa de sesiones, decisiones y resultados a través de una base de datos PostgreSQL correctamente administrada.

1.3. Alcance del informe

Este informe documenta exclusivamente la administración de la base de datos PostgreSQL 16 del proyecto SBVIA, cubriendo los cinco criterios evaluados en la rúbrica del Examen Final de la asignatura *Administración de Bases de Datos*:

1. Diseño de la base de datos y carga del millón de registros.
2. Implementación y gestión de respaldos.
3. Auditoría de operaciones de la base de datos.
4. Control de usuarios, roles y permisos.
5. Índices y optimización de consultas (EXPLAIN ANALYZE).

Capítulo 2

Arquitectura General del Sistema

2.1. Visión general

SBVIA implementa una arquitectura de microservicios distribuida organizada en cuatro capas principales que se comunican a través de una red Docker interna:

Tabla 2.1: Componentes principales del sistema SBVIA

Capa	Tecnología	Responsabilidad
Frontend	Angular 17	Interfaz web reactiva: autenticación, dashboard, escenarios, reportes y perfil del participante. Consume la API REST del backend.
Backend API	Spring Boot 3.2	Expone endpoints REST con autenticación JWT en cookies HttpOnly. Gestiona lógica de negocio, procedimientos almacenados y cache.
Base de datos	PostgreSQL 16	Persistencia durable de toda la información: usuarios, escenarios, simulaciones, decisiones, métricas y auditoría.
Caché	Redis 7	Caché de lecturas frecuentes (escenarios, sesiones). Reduce carga sobre PostgreSQL en consultas repetitivas.

2.2. Módulos principales del backend

- **Módulo de Autenticación y Seguridad:** Gestión de JWT, cookies seguras (HttpOnly + Secure + SameSite), filtros de autorización por rol (ADMINISTRADOR, INSTRUCTOR, PARTICIPANTE) y limitación de intentos de login.
- **Módulo de Escenarios:** CRUD de escenarios viales con caché Redis. Cada escenario define tipo de vía, clima, densidad de tráfico y nivel de dificultad.
- **Módulo de Simulaciones:** Registro de sesiones de entrenamiento, intentos (simulaciones), eventos viales, decisiones del conductor y cálculo automático de puntaje.

- **Módulo de Reportes y Métricas:** Invoca procedimientos almacenados PostgreSQL para generar reportes multi-tabla eficientes sin SQL dinámico.
- **Módulo de Auditoría:** Registra automáticamente cada intento de autenticación (exitoso o fallido) en la tabla `historial_acceso` a través de triggers PL/pgSQL.

2.3. Flujo de una simulación

1. El participante se autentica y su acceso queda registrado en `historial_acceso` (trigger de auditoría).
2. Selecciona un escenario (leído desde caché Redis o PostgreSQL).
3. Se crea una `sesion_entrenamiento` y una `simulacion`.
4. Durante la simulación se registran `evento_vial`, `decision` y `progreso_simulacion`.
5. Al finalizar, el trigger `trg_recalcular_puntaje_infraccion` calcula automáticamente el `puntaje_final`.
6. Se genera `comportamiento_vial`, `retroalimentacion` y `evaluacion_ia`.
7. El participante puede consultar su historial desde `vw_historial_simulaciones`.

2.4. Relación con la base de datos

Spring Boot accede a PostgreSQL mediante una **estrategia híbrida**:

- **Spring Data JPA / Hibernate** para operaciones CRUD elementales (usuarios, escenarios, vehículos).
- **Procedimientos Almacenados** (db/procs/) invocados mediante `@Procedure` de JPA para operaciones analíticas, reportes y cálculos agregados multi-tabla.

Los 12 procedimientos almacenados documentados en `docs/basedatos/CATALOGO-SP.md` incluyen: `sp_reporte_simulacion`, `sp_calcular_promedio_usuario`, `sp_reporte_actividad_diaria`, `sp_actualizar_usuarios_inactivos`, `sp_validar_escenario` y `sp_generar_codigo_certificado`.

Capítulo 3

Diseño de la Base de Datos y Millón de Registros

3.1. Descripción del modelo relacional

La base de datos **sbvia** está compuesta por **21 tablas**, organizadas en dos grupos funcionales:

Tablas de catálogo: rol, estado_usuario, estado_simulacion, tipo_via, tipo_clima, nivel_dificultad, nivel_gravedad, tipo_evento, tipo_metrica, tipo_vehiculo, modelo_ia, regla_transito.

Tablas transaccionales: usuario, vehiculo, escenario, sesion_entrenamiento, simulacion, evento_vial, decision, progreso_simulacion, comportamiento_vial, infraccion, metrica_desempeno, retroalimentacion, evaluacion_ia, historial_acceso.

3.2. Estructura de tablas principales

3.2.1. Tabla usuario

```
1 CREATE TABLE public.usuario (  
2     id_usuario          bigint      NOT NULL,  
3     nombres             varchar(100) NOT NULL,  
4     apellidos           varchar(100) NOT NULL,  
5     correo              varchar(150) NOT NULL,  
6     nombre_usuario      varchar(60)  NOT NULL,  
7     contrasena_hash     varchar(255) NOT NULL,  
8     telefono            varchar(20),  
9     fecha_nacimiento    date,  
10    fecha_registro      timestamp DEFAULT CURRENT_TIMESTAMP NOT NULL,  
11    ultimo_acceso       timestamp,  
12    intentos_fallidos    integer     DEFAULT 0      NOT NULL,  
13    cuenta_bloqueada    boolean     DEFAULT false NOT NULL,  
14    id_rol               integer     NOT NULL,  
15    id_estado_usuario    integer     NOT NULL,  
16    CONSTRAINT chk_usuario_correo  
17    CHECK (correo ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'),
```



```

18  CONSTRAINT chk_usuario_contrasena_hash
19      CHECK (length(btrim(contrasena_hash)) >= 20),
20  CONSTRAINT chk_usuario_nombre_usuario
21      CHECK (length(btrim(nombre_usuario)) >= 4)
22  );

```

Listing 3.1: Definición de la tabla usuario

3.2.2. Tabla simulacion

```

1  CREATE TABLE public.simulacion (
2      id_simulacion          bigint NOT NULL,
3      id_sesion              bigint NOT NULL,
4      id_usuario             bigint NOT NULL,
5      id_escenario           bigint NOT NULL,
6      id_vehiculo            bigint NOT NULL,
7      id_estado_simulacion   integer NOT NULL,
8      numero_intento         integer NOT NULL,
9      fecha_inicio           timestamp DEFAULT CURRENT_TIMESTAMP NOT NULL,
10     fecha_fin               timestamp,
11     porcentaje_progreso     numeric(5,2) DEFAULT 0 NOT NULL,
12     puntaje_final           numeric(8,2),
13     duracion_segundos       integer,
14     completada              boolean DEFAULT false NOT NULL,
15     CONSTRAINT chk_simulacion_progreso
16         CHECK (porcentaje_progreso >= 0 AND porcentaje_progreso <= 100),
17     CONSTRAINT chk_simulacion_puntaje
18         CHECK (puntaje_final IS NULL OR puntaje_final >= 0),
19     CONSTRAINT chk_simulacion_intento
20         CHECK (numero_intento > 0)
21 );

```

Listing 3.2: Definición de la tabla simulacion

3.3. Relaciones e integridad referencial

3.4. Restricciones CHECK implementadas

Se implementaron **20 restricciones CHECK** a nivel de motor de base de datos:

```

1  -- Clasificacion de comportamiento vial: valores permitidos
2  CONSTRAINT chk_comportamiento_clasificacion
3      CHECK (clasificacion IN
4          ('EXCELENTE', 'BUENO', 'REGULAR', 'RIESGOSO', 'CRITICO')),
5
6  -- Resultado de decision: valores controlados
7  CONSTRAINT chk_decision_resultado
8      CHECK (resultado IN
9          ('CORRECTA', 'PARCIAL', 'INCORRECTA', 'NO_REALIZADA')),
10
11 -- Densidad de trafico en escenario

```

Tabla 3.1: Relaciones clave del modelo SBVIA

Tabla Hijo	Tabla Padre	Tipo	Comportamiento
usuario	rol	N:1	ON DELETE RESTRICT
usuario	estado_usuario	N:1	ON DELETE RESTRICT
simulacion	sesion_entrenamiento	N:1	ON DELETE CASCADE
simulacion	usuario	N:1	ON DELETE RESTRICT
simulacion	escenario	N:1	ON DELETE RESTRICT
evento_vial	simulacion	N:1	ON DELETE CASCADE
decision	simulacion	N:1	ON DELETE CASCADE
infraccion	simulacion	N:1	ON DELETE CASCADE
infraccion	regla_transito	N:1	ON DELETE RESTRICT
comportamiento_vial	simulacion	1:1	ON DELETE CASCADE
historial_acceso	usuario	N:1	ON DELETE CASCADE

```

12 CONSTRAINT chk_escenario_densidad
13     CHECK (densidad_trafico IN ('BAJA', 'MEDIA', 'ALTA', 'MUY_ALTA')),
14
15 -- Nivel de dificultad: rango 1 a 10
16 CONSTRAINT chk_nivel_dificultad_valor
17     CHECK (valor >= 1 AND valor <= 10),
18
19 -- Penalizacion no puede ser negativa
20 CONSTRAINT chk_infraccion_penalizacion
21     CHECK (penalizacion_aplicada >= 0);

```

Listing 3.3: Ejemplos de restricciones CHECK

3.5. Triggers de automatización

Tabla 3.2: Triggers implementados en SBVIA

Trigger	Evento	Propósito
trg_actualizar_ultimo_acceso	historial_acceso / INSERT	Actualiza timestamp de acceso e incrementa contador de fallos.
trg_recalcular_puntaje_infraccion	infraccion / INSERT, UPDATE, DELETE	Recalcula puntaje final de simulación.
trg_finalizar_simulacion_progreso	progreso_simulacion / INSERT, UPDATE, DATE	Marca simulación como COMPLETA al 100%.
trg_sincronizar_progreso	progreso_simulacion / INSERT	Sincroniza porcentaje en tabla simulacion.

```

1 CREATE FUNCTION public.fn_recalcular_puntaje_simulacion()
2 RETURNS trigger LANGUAGE plpgsql AS $$
3 DECLARE
4     v_id_simulacion BIGINT;
5     v_penalizacion NUMERIC(10,2);
6 BEGIN

```

```

7      v_id_simulacion := COALESCE(NEW.id_simulacion, OLD.id_simulacion);
8      SELECT COALESCE(SUM(penalizacion_aplicada), 0)
9          INTO v_penalizacion
10         FROM infraccion
11        WHERE id_simulacion = v_id_simulacion;
12      UPDATE simulacion
13         SET puntaje_final = GREATEST(0, 100 - v_penalizacion)
14        WHERE id_simulacion = v_id_simulacion;
15      RETURN COALESCE(NEW, OLD);
16 END; $$;

```

Listing 3.4: Función del trigger de recálculo de puntaje

3.6. Vistas del sistema

Se implementaron **5 vistas** para facilitar consultas frecuentes: `vw_historial_simulaciones`, `vw_infracciones_detalladas`, `vw_metricas_simulacion`, `vw_ranking_participantes`, `vw_retroalimentacion`.

```

1 CREATE VIEW public.vw_historial_simulaciones AS
2 SELECT s.id_simulacion,
3        concat(u.nombres, ' ', u.apellidos) AS participante,
4        e.nombre AS escenario,
5        v.nombre AS vehiculo,
6        es.nombre AS estado,
7        s.numero_intento,
8        s.fecha_inicio,
9        s.puntaje_final,
10       s.completada
11 FROM simulacion s
12 JOIN usuario u ON u.id_usuario = s.id_usuario
13 JOIN escenario e ON e.id_escenario = s.id_escenario
14 JOIN vehiculo v ON v.id_vehiculo = s.id_vehiculo
15 JOIN estado_simulacion es ON es.id_estado_simulacion = s.id_estado_simulacion
16 ;

```

Listing 3.5: Vista de historial de simulaciones

3.7. Generación del millón de registros

3.7.1. Estrategia de carga masiva

El script `01_generar_millon_registros.sql` genera datos coherentes usando `generate_series()` de PostgreSQL. Desactiva triggers con `SET session_replication_role = 'replica'` para eficiencia en la carga.

```

1 SET session_replication_role = 'replica';
2
3 DO $$ DECLARE
4     v_id_rol_participante INTEGER;
5     v_id_escenario        BIGINT;
6 BEGIN

```

```

7      -- 1. Usuarios (2,000)
8      INSERT INTO usuario (nombres, apellidos, correo, nombre_usuario,
9                          contrasena_hash, id_rol, id_estado_usuario, ...)
10     SELECT 'CargaMasiva_Nombres_' || i,
11            'CargaMasiva_Apellidos_' || i,
12            'carga.usuario' || i || '@sbvia.edu.ec',
13            'cargau' || i,
14            '$2a$10$EXAMENDEMOHASHSEGUR00000000000000000000',
15            v_id_rol_participante, v_id_estado_activo, ...
16     FROM generate_series(1, 2000) AS i;
17
18     -- 2. Sesiones de entrenamiento (10,000 = 5 por usuario)
19     INSERT INTO sesion_entrenamiento (id_usuario, fecha_inicio, estado, ...)
20     SELECT u.id_usuario, u.fecha_registro + (s.i || ' hours')::INTERVAL,
21            'FINALIZADA', ...
22     FROM usuario u CROSS JOIN generate_series(1, 5) AS s(i)
23     WHERE u.correo LIKE 'carga.usuario%';
24
25     -- 3. Simulaciones (30,000 = 3 por sesion)
26     -- 4. Eventos viales (150,000 = 5 por simulacion)
27     -- 5. Decisiones (600,000 = 20 por simulacion)
28     -- 6. Progresos (300,000 = 10 por simulacion)
29     -- 7-12. Metricas, comportamientos, infracciones,
30     --      retroalimentacion, evaluacion_ia, historial_acceso
31 END $$;
32
33 SET session_replication_role = 'origin';
34 VACUUM ANALYZE public.simulacion;
35 VACUUM ANALYZE public.decision;

```

Listing 3.6: Fragmento del script de carga masiva

3.7.2. Distribución de registros generados

Tabla 3.3: Distribución de registros por tabla tras la carga masiva

Tabla	Fórmula	Registros
decision	30,000 sim × 20	600,000
progreso_simulacion	30,000 sim × 10	300,000
evento_vial	30,000 sim × 5	150,000
historial_acceso	2,000 usr × 50	100,000
infraccion	30,000 sim × 2	60,000
simulacion	10,000 ses × 3	30,000
metrica_desempeno	30,000 sim × 1	30,000
comportamiento_vial	30,000 sim × 1	30,000
retroalimentacion	30,000 sim × 1	30,000
evaluacion_ia	30,000 sim × 1	30,000
sesion_entrenamiento	2,000 usr × 5	10,000
usuario	generados directos	2,000
TOTAL		1,372,000

3.7.3. Consulta de verificación

```

1  -- Conteo total por tabla transaccional
2  SELECT 'usuario' AS tabla, COUNT(*) AS registros
3  FROM public.usuario
4  UNION ALL SELECT 'sesion_entrenamiento', COUNT(*) FROM public.
   sesion_entrenamiento
5  UNION ALL SELECT 'simulacion', COUNT(*) FROM public.simulacion
6  UNION ALL SELECT 'evento_vial', COUNT(*) FROM public.evento_vial
7  UNION ALL SELECT 'decision', COUNT(*) FROM public.decision
8  UNION ALL SELECT 'progreso_simulacion', COUNT(*) FROM public.
   progreso_simulacion
9  UNION ALL SELECT 'metrica_desempeno', COUNT(*) FROM public.
   metrica_desempeno
10 UNION ALL SELECT 'comportamiento_vial', COUNT(*) FROM public.
   comportamiento_vial
11 UNION ALL SELECT 'infraccion', COUNT(*) FROM public.infraccion
12 UNION ALL SELECT 'retroalimentacion', COUNT(*) FROM public.
   retroalimentacion
13 UNION ALL SELECT 'evaluacion_ia', COUNT(*) FROM public.evaluacion_ia
14 UNION ALL SELECT 'historial_acceso', COUNT(*) FROM public.
   historial_acceso
15 ORDER BY registros DESC;
16
17 -- Tamano total de la base de datos
18 SELECT pg_size_pretty(pg_database_size(current_database())) AS tamano_bd,
19        (SELECT SUM(n_live_tup) FROM pg_stat_user_tables) AS
   total_filas_estimadas;

```

Listing 3.7: Script 02_verificar_millon_registros.sql

Capítulo 4

Implementación y Gestión de Respaldos

4.1. Herramienta: pg_dump

PostgreSQL provee la utilidad nativa `pg_dump` para generar respaldos consistentes sin bloquear operaciones concurrentes. El formato `custom` (`-Fc`) permite compresión y restauración selectiva con `pg_restore`.

4.2. Generación del respaldo

```
1  # Respaldo completo en formato custom (comprimido, nivel 9)
2  pg_dump \
3    --host=localhost \
4    --port=5432 \
5    --username=sbvia_admin \
6    --dbname=sbvia \
7    --format=custom \
8    --compress=9 \
9    --file=respaldo_sbvia_$(date +%Y%m%d_%H%M%S).backup \
10   --verbose
11
12  # Respaldo solo del esquema (sin datos)
13  pg_dump --schema-only -U sbvia_admin -d sbvia -f schema_sbvia.sql
14
15  # Respaldo solo de los datos (sin esquema)
16  pg_dump --data-only -U sbvia_admin -d sbvia -Fc -f datos_sbvia.backup
```

Listing 4.1: Comando de generación de respaldo completo

4.3. Respaldo desde contenedor Docker

```
1  # Ejecutar pg_dump dentro del contenedor de PostgreSQL
2  docker compose exec db pg_dump \
```

```

3  -U sbvia_user -d sbvia -Fc \
4  -f /var/lib/postgresql/backups/sbvia_$(date +%Y%m%d).backup
5
6  # Copiar el backup al equipo local
7  docker compose cp \
8    db:/var/lib/postgresql/backups/. \
9    ./backups/

```

Listing 4.2: Respaldo desde Docker Compose

4.4. Restauración

```

1  # 1. Crear la base de datos destino
2  createdb -U sbvia_admin sbvia_restored
3
4  # 2. Restaurar desde el archivo .backup
5  pg_restore \
6    --host=localhost --port=5432 \
7    --username=sbvia_admin \
8    --dbname=sbvia_restored \
9    --verbose --clean --if-exists \
10   respaldo_sbvia_20260908.backup
11
12 # 3. Verificar la restauracion
13 psql -U sbvia_admin -d sbvia_restored \
14   -c "SELECT COUNT(*) FROM simulacion;"

```

Listing 4.3: Restauración completa con pg_restore

4.5. Script de respaldo automatizado

```

1  #!/bin/bash
2  BACKUP_DIR="/opt/sbvia/backups"
3  FECHA=$(date +%Y%m%d_%H%M%S)
4  ARCHIVO="$BACKUP_DIR/sbvia_$FECHA.backup"
5  RETENCION_DIAS=7
6
7  mkdir -p "$BACKUP_DIR"
8
9  pg_dump -h localhost -p 5432 \
10   -U sbvia_admin -d sbvia \
11   -Fc --compress=9 --file="$ARCHIVO"
12
13 if [ $? -eq 0 ]; then
14   echo "[OK] Respaldo: $ARCHIVO ($(du -h $ARCHIVO | cut -f1))"
15 else
16   echo "[ERROR] Fallo en la generacion del respaldo." >&2
17   exit 1
18 fi
19
20 # Eliminar backups con mas de 7 dias

```

```
21 find "$BACKUP_DIR" -name "sbvia_*.backup" -mtime +$RETENCION_DIAS -delete
22 echo "[OK] Backups anteriores a $RETENCION_DIAS dias eliminados."
```

Listing 4.4: Script de respaldo con retención (backup_sbvia.sh)

La evidencia del respaldo se encuentra en el archivo `database/current/schema.sql` (62,888 bytes), generado con `pg_dump` y que contiene el esquema completo de la base de datos, incluyendo funciones, triggers, tablas, secuencias, índices y claves foráneas.

Capítulo 5

Auditoría de Operaciones de la Base de Datos

5.1. Sistema de auditoría de dos niveles

SBVIA implementa auditoría en dos niveles diferenciados:

5.1.1. Nivel 1: Auditoría de accesos

La tabla `historial_acceso` registra todos los intentos de autenticación (exitosos y fallidos):

```
1 CREATE TABLE public.historial_acceso (  
2     id_historial_acceso bigint NOT NULL,  
3     id_usuario          bigint NOT NULL,  
4     fecha_hora          timestamp DEFAULT CURRENT_TIMESTAMP NOT NULL,  
5     direccion_ip        inet,  
6     dispositivo         varchar(150),  
7     navegador           varchar(100),  
8     acceso_exitoso      boolean NOT NULL,  
9     detalle             varchar(255)  
10 );  
11 -- FK: id_usuario -> usuario ON DELETE CASCADE  
12 COMMENT ON TABLE public.historial_acceso IS  
13     'Registro historico de accesos exitosos y fallidos.';
```

Listing 5.1: Definición de la tabla `historial_acceso`

5.1.2. Nivel 2: Trigger de auditoría automática

```
1 CREATE FUNCTION public.fn_actualizar_ultimo_acceso()  
2 RETURNS trigger LANGUAGE plpgsql AS $$  
3 BEGIN  
4     IF NEW.acceso_exitoso = TRUE THEN  
5         -- Login exitoso: actualizar timestamp y resetear contador  
6         UPDATE usuario
```

```

7      SET ultimo_acceso      = NEW.fecha_hora,
8          intentos_fallidos = 0
9      WHERE id_usuario = NEW.id_usuario;
10  ELSE
11      -- Login fallido: incrementar contador (deteccion de ataques)
12      UPDATE usuario
13      SET intentos_fallidos = intentos_fallidos + 1
14      WHERE id_usuario = NEW.id_usuario;
15  END IF;
16  RETURN NEW;
17 END; $$;
18
19 CREATE TRIGGER trg_actualizar_ultimo_acceso
20 AFTER INSERT ON public.historial_acceso
21 FOR EACH ROW
22 EXECUTE FUNCTION public.fn_actualizar_ultimo_acceso();

```

Listing 5.2: Trigger y función de auditoría de accesos

5.2. Consultas de auditoría

```

1  -- 1. Accesos de las ultimas 24 horas
2  SELECT concat(u.nombres, ' ', u.apellidos) AS usuario,
3          ha.fecha_hora, ha.direccion_ip,
4          ha.navegador, ha.acceso_exitoso, ha.detalle
5  FROM historial_acceso ha
6  JOIN usuario u ON u.id_usuario = ha.id_usuario
7  WHERE ha.fecha_hora >= NOW() - INTERVAL '24 hours'
8  ORDER BY ha.fecha_hora DESC;
9
10 -- 2. Detectar posibles ataques (multiples fallos en 1 hora)
11 SELECT u.correo, u.nombres, u.intentos_fallidos,
12        u.cuenta_bloqueada,
13        COUNT(*) AS intentos_fallidos_recientes
14 FROM usuario u
15 JOIN historial_acceso ha ON ha.id_usuario = u.id_usuario
16 WHERE ha.acceso_exitoso = FALSE
17       AND ha.fecha_hora >= NOW() - INTERVAL '1 hour'
18 GROUP BY u.id_usuario, u.correo, u.nombres,
19          u.intentos_fallidos, u.cuenta_bloqueada
20 HAVING COUNT(*) >= 3
21 ORDER BY intentos_fallidos_recientes DESC;
22
23 -- 3. Estadísticas de acceso diarias (ultimos 30 dias)
24 SELECT DATE(fecha_hora) AS fecha,
25        COUNT(*) FILTER (WHERE acceso_exitoso) AS exitosos,
26        COUNT(*) FILTER (WHERE NOT acceso_exitoso) AS fallidos,
27        COUNT(DISTINCT id_usuario) AS usuarios_unicos
28 FROM historial_acceso
29 GROUP BY DATE(fecha_hora)
30 ORDER BY fecha DESC LIMIT 30;

```

Listing 5.3: Consultas para revisar el historial de auditoría

5.3. Auditoría DML extendida

Para operaciones DML críticas se puede implementar una tabla de auditoría general:

```
1 CREATE TABLE public.auditoria_operaciones (  
2     id_auditoria      bigserial PRIMARY KEY,  
3     tabla_afectada    varchar(60) NOT NULL,  
4     operacion         varchar(10) NOT NULL, -- INSERT, UPDATE, DELETE  
5     usuario_db        varchar(60) DEFAULT current_user,  
6     fecha_hora        timestamp  DEFAULT CURRENT_TIMESTAMP,  
7     datos_anteriores  jsonb,  
8     datos_nuevos      jsonb  
9 );  
10  
11 CREATE OR REPLACE FUNCTION fn_auditar_operacion()  
12 RETURNS trigger LANGUAGE plpgsql AS $$  
13 BEGIN  
14     IF TG_OP = 'INSERT' THEN  
15         INSERT INTO auditoria_operaciones  
16             (tabla_afectada, operacion, datos_nuevos)  
17             VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(NEW));  
18         RETURN NEW;  
19     ELSIF TG_OP = 'UPDATE' THEN  
20         INSERT INTO auditoria_operaciones  
21             (tabla_afectada, operacion, datos_anteriores, datos_nuevos)  
22             VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(OLD), to_jsonb(NEW));  
23         RETURN NEW;  
24     ELSIF TG_OP = 'DELETE' THEN  
25         INSERT INTO auditoria_operaciones  
26             (tabla_afectada, operacion, datos_anteriores)  
27             VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(OLD));  
28         RETURN OLD;  
29     END IF;  
30 END; $$;
```

Listing 5.4: Tabla de auditoría DML genérica

Capítulo 6

Control de Usuarios, Roles y Permisos

6.1. Roles del sistema (nivel de aplicación)

El modelo RBAC de SBVIA define tres roles en la tabla rol:

Tabla 6.1: Roles del sistema SBVIA

Rol	Permisos
ADMINISTRADOR	Acceso total: gestión de usuarios, catálogos, escenarios, vehículos y reportes. Puede bloquear cuentas.
INSTRUCTOR	Puede crear/gestionar escenarios, ver reportes de todos los participantes y generar retroalimentación.
PARTICIPANTE	Solo puede ejecutar simulaciones, ver su propio historial y recibir retroalimentación personal.

6.2. Usuarios PostgreSQL con permisos diferenciados

```
1  -- =====
2  -- SBVIA - Control de Usuarios y Permisos PostgreSQL
3  -- =====
4
5  -- 1. CREAR USUARIOS DE BASE DE DATOS
6  CREATE USER sbvia_admin
7      WITH PASSWORD 'Admin@SBVIA2026!' CREATEDB LOGIN;
8
9  CREATE USER sbvia_app
10     WITH PASSWORD 'App@SBVIA2026!' LOGIN;
11
12  CREATE USER sbvia_readonly
13     WITH PASSWORD 'Read@SBVIA2026!' LOGIN;
14
15  -- 2. CREAR ROLES AGRUPADORES
16  CREATE ROLE sbvia_escritura;
```

```

17 CREATE ROLE sbvia_lectura;
18
19 -- 3. PERMISOS AL ROL DE ESCRITURA (sbvia_app)
20 GRANT CONNECT ON DATABASE sbvia TO sbvia_escritura;
21 GRANT USAGE ON SCHEMA public TO sbvia_escritura;
22 GRANT SELECT, INSERT, UPDATE, DELETE
23     ON ALL TABLES IN SCHEMA public TO sbvia_escritura;
24 GRANT USAGE, SELECT
25     ON ALL SEQUENCES IN SCHEMA public TO sbvia_escritura;
26
27 -- 4. PERMISOS AL ROL DE LECTURA (sbvia_readonly)
28 GRANT CONNECT ON DATABASE sbvia TO sbvia_lectura;
29 GRANT USAGE ON SCHEMA public TO sbvia_lectura;
30 GRANT SELECT ON ALL TABLES IN SCHEMA public TO sbvia_lectura;
31
32 -- 5. ASIGNAR ROLES A USUARIOS
33 GRANT sbvia_escritura TO sbvia_app;
34 GRANT sbvia_lectura TO sbvia_readonly;
35 GRANT ALL PRIVILEGES ON DATABASE sbvia TO sbvia_admin;
36
37 -- 6. RESTRINGIR ACCESOS NO NECESARIOS
38 REVOKE CREATE ON SCHEMA public FROM PUBLIC;
39 REVOKE ALL ON DATABASE sbvia FROM PUBLIC;

```

Listing 6.1: Script 03_usuarios_rol_permisos.sql

6.3. Demostración de restricción de acceso

```

1 -- Conectado como: sbvia_readonly
2
3 -- LECTURA (PERMITIDO)
4 SELECT id_usuario, nombres, correo
5 FROM public.usuario LIMIT 5;
6 -- Resultado: 5 filas devueltas correctamente
7
8 -- ESCRITURA (DENEGADO)
9 INSERT INTO public.usuario (nombres, apellidos, correo, ...)
10 VALUES ('Prueba', 'Test', 'prueba@test.com', ...);
11 -- ERROR: permiso denegado a relacion usuario
12 -- (permission denied for relation usuario)
13
14 -- ELIMINACION (DENEGADO)
15 DELETE FROM public.usuario WHERE id_usuario = 1;
16 -- ERROR: permiso denegado a relacion usuario

```

Listing 6.2: sbvia_readonly: lectura permitida, escritura denegada

```

1 -- Conectado como: sbvia_app
2
3 -- INSERCIÓN (PERMITIDO)
4 INSERT INTO public.historial_acceso
5     (id_usuario, direccion_ip, acceso_exitoso, detalle)
6 VALUES (1, '192.168.1.10', TRUE, 'Login de prueba');

```

```
7  -- INSERT 0 1 (exito)
8
9  -- DROP (DENEGADO)
10 DROP TABLE public.usuario;
11 -- ERROR: permiso denegado a relacion usuario
12 -- Solo sbvia_admin puede ejecutar DDL
```

Listing 6.3: sbvia_app: operaciones DML permitidas, DDL denegado

6.4. Vista segura para columnas sensibles

```
1  -- Vista que excluye la contraseña_hash
2 CREATE VIEW public.vw_usuarios_publico AS
3 SELECT id_usuario, nombres, apellidos, correo,
4        nombre_usuario, fecha_registro,
5        ultimo_acceso, cuenta_bloqueada,
6        id_rol, id_estado_usuario
7 FROM public.usuario;
8
9  -- Solo dar acceso a la vista (no a la tabla base)
10 GRANT SELECT ON public.vw_usuarios_publico TO sbvia_lectura;
11 REVOKE SELECT ON public.usuario FROM sbvia_lectura;
```

Listing 6.4: Protección de columna contraseña_hash mediante vista

Capítulo 7

Índices y Optimización de Consultas

7.1. Índices implementados

Se implementaron **13 índices B-Tree compuestos** sobre las tablas transaccionales más consultadas:

Tabla 7.1: Índices implementados con justificación técnica

Índice	Tabla / Columnas	Justificación
idx_simulacion_usuario_fecha	simulacion (id_usuario, fecha_inicio DESC)	Consulta más frecuente: historial de un usuario.
idx_simulacion_escenario	simulacion (id_escenario)	FK sin índice implícito, usada en JOINS.
idx_simulacion_estado	simulacion (id_estado_simulacion)	Filtros por estado (COMPLETADA) muy frecuentes.
idx_decision_simulacion_fecha	decision (id_simulacion, fecha_hora)	Recuperar decisiones cronológicas por simulación.
idx_evento_simulacion_fecha	evento_vial (id_simulacion, fecha_hora)	Reconstrucción temporal de eventos.
idx_infraccion_simulacion_fecha	infraccion (id_simulacion, fecha_hora)	Vista vw_infracciones_detalladas.
idx_historial_acceso_usuario_fecha	historial_acceso (id_usuario, fecha_hora DESC)	Auditoría: accesos recientes de un usuario.

continúa...

Índice	Tabla / Columnas	Justificación
idx_metrica_simulacion_tipo	metrica_desempeno (id_simulacion, id_tipo_metrica)	JOIN compuesto en vw_metricas_simulacion.
idx_progreso_simulacion_fecha	progreso_simulacion (id_simulacion, fecha_hora)	Avance cronológico de una simulación.
idx_evaluacion_ia_simulacion	evaluacion_ia (id_simulacion)	Tabla de alta escritura, accedida en reportes.
idx_sesion_usuario_fecha	sesion_entrenamiento (id_usuario, fecha_inicio)	Historial de sesiones por usuario.
idx_usuario_rol	usuario (id_rol)	Listar todos los participantes por rol.
idx_usuario_estado	usuario (id_estado_usuario)	Filtrado de usuarios activos en panel admin.

7.2. Creación de índices

```

1  -- Indices en tabla simulacion (la mas consultada)
2  CREATE INDEX idx_simulacion_usuario_fecha
3      ON public.simulacion USING btree (id_usuario, fecha_inicio DESC);
4
5  CREATE INDEX idx_simulacion_escenario
6      ON public.simulacion USING btree (id_escenario);
7
8  CREATE INDEX idx_simulacion_estado
9      ON public.simulacion USING btree (id_estado_simulacion);
10
11 -- Indices en tablas de alta cardinalidad
12 CREATE INDEX idx_decision_simulacion_fecha
13     ON public.decision USING btree (id_simulacion, fecha_hora);
14
15 CREATE INDEX idx_evento_simulacion_fecha
16     ON public.evento_vial USING btree (id_simulacion, fecha_hora);
17
18 CREATE INDEX idx_historial_acceso_usuario_fecha
19     ON public.historial_acceso USING btree (id_usuario, fecha_hora DESC);
20
21 -- Indice en metricas
22 CREATE INDEX idx_metrica_simulacion_tipo
23     ON public.metrica_desempeno USING btree (id_simulacion, id_tipo_metrica);

```

Listing 7.1: Creación de los índices de rendimiento

7.3. Análisis EXPLAIN ANALYZE

7.3.1. Consulta 1: Historial de simulaciones de un usuario

Consulta más frecuente del sistema, ejecutada en el dashboard del participante.

Sin índice (antes)

```

1 EXPLAIN ANALYZE
2 SELECT s.id_simulacion, s.puntaje_final, s.fecha_inicio,
3        e.nombre AS escenario
4 FROM simulacion s
5 JOIN escenario e ON e.id_escenario = s.id_escenario
6 WHERE s.id_usuario = 42
7 ORDER BY s.fecha_inicio DESC;
8
9 /*
10 Sort  (cost=18423.55..18453.67 rows=12048 width=48)
11       (actual time=284.321..284.399 rows=15 loops=1)
12   Sort Key: s.fecha_inicio DESC
13   -> Hash Join (cost=4821.00..17612.11 rows=12048 width=48)
14         (actual time=180.312..284.198 rows=15 loops=1)
15       -> Seq Scan on simulacion s
16             (cost=0.00..12148.00 rows=30000 width=36)
17             (actual time=0.015..165.221 rows=30000 loops=1)
18           Filter: (id_usuario = 42)
19           Rows Removed by Filter: 29985
20     ...
21 Planning Time: 2.1 ms
22 Execution Time: 284.5 ms      <-- LENTO (Seq Scan de 30,000 filas)
23 */

```

Listing 7.2: EXPLAIN ANALYZE sin índice — Sequential Scan

Con índice (después)

```

1 CREATE INDEX idx_simulacion_usuario_fecha
2   ON simulacion USING btree (id_usuario, fecha_inicio DESC);
3
4 EXPLAIN ANALYZE
5 SELECT s.id_simulacion, s.puntaje_final, s.fecha_inicio,
6        e.nombre AS escenario
7 FROM simulacion s
8 JOIN escenario e ON e.id_escenario = s.id_escenario
9 WHERE s.id_usuario = 42
10 ORDER BY s.fecha_inicio DESC;
11
12 /*
13 Sort  (cost=48.12..48.15 rows=15 width=48)
14       (actual time=0.412..0.418 rows=15 loops=1)
15   Sort Key: s.fecha_inicio DESC
16   -> Nested Loop (cost=0.57..47.84 rows=15 width=48)

```

```
17      (actual time=0.051..0.388 rows=15 loops=1)
18  ->  Index Scan using idx_simulacion_usuario_fecha
19      on simulacion s
20      (cost=0.29..12.38 rows=15 width=36)
21      (actual time=0.031..0.188 rows=15 loops=1)
22      Index Cond: (id_usuario = 42)
23  ...
24  Planning Time: 1.8 ms
25  Execution Time: 0.52 ms      <-- 546x MAS RAPIDO
26  */
```

Listing 7.3: EXPLAIN ANALYZE con índice — Index Scan

Tabla 7.2: Comparación EXPLAIN ANALYZE — Consulta historial por usuario

Métrica	Sin índice	Con índice	Mejora
Método de acceso	Seq Scan	Index Scan	—
Filas leídas	30,000	15	2,000× menos
Costo estimado	18,423.55	48.12	≈382× menor
Tiempo ejecución	284.5 ms	0.52 ms	546× más rápido

7.3.2. Consulta 2: Auditoría de accesos recientes

```
1 CREATE INDEX idx_historial_acceso_usuario_fecha
2   ON historial_acceso USING btree (id_usuario, fecha_hora DESC);
3
4 EXPLAIN ANALYZE
5 SELECT fecha_hora, direccion_ip, acceso_exitoso, detalle
6 FROM historial_acceso
7 WHERE id_usuario = 42
8    AND fecha_hora >= NOW() - INTERVAL '7 days'
9 ORDER BY fecha_hora DESC;
10
11 /*
12 -- SIN indice: Seq Scan sobre 100,000 filas ~ 320 ms
13 -- CON indice:
14 Index Scan using idx_historial_acceso_usuario_fecha
15   on historial_acceso
16   Index Cond: ((id_usuario = 42)
17     AND (fecha_hora >= (now() - '7 days'::interval)))
18   Rows Removed by Filter: 1
19 Execution Time: 0.20 ms      <-- >1,600x mas rapido sobre 100,000 filas
20 */
```

Listing 7.4: EXPLAIN ANALYZE auditoría con índice compuesto

7.4. Glosario de campos EXPLAIN ANALYZE

7.5. Verificación del uso de índices

Tabla 7.3: Comparación EXPLAIN ANALYZE — Consulta auditoría de accesos

Métrica	Sin índice	Con índice	Mejora
Método de acceso	Seq Scan	Index Scan	—
Filas leídas	100,000	≈50	2,000× menos
Tiempo ejecución	≈320 ms	0.20 ms	>1,600× más rápido

Tabla 7.4: Interpretación de campos clave en EXPLAIN ANALYZE

Campo	Significado
cost=X..Y	Costo estimado. X = startup cost, Y = total cost (en unidades de páginas de disco).
actual time=X..Y	Tiempo real en milisegundos. X = primer resultado, Y = todos los resultados.
rows=N	Filas estimadas que devolverá el nodo.
Seq Scan	Recorre la tabla completa. Ineficiente con filtros selectivos en tablas grandes.
Index Scan	Usa el índice B-Tree para localizar directamente las filas. Eficiente con alta selectividad.
Index Only Scan	Los datos requeridos están en el índice; no accede a la tabla (cobertura total).
Rows Removed by Filter	Filas leídas pero descartadas por el WHERE. Alto valor indica necesidad de índice.

```

1  -- Ver todos los índices y su tamaño
2  SELECT schemaname, tablename, indexname,
3         pg_size_pretty(pg_relation_size(indexrelid)) AS tamaño
4  FROM pg_stat_user_indexes
5  ORDER BY pg_relation_size(indexrelid) DESC;
6
7  -- Verificar cuantas veces se usa cada índice
8  SELECT relname AS tabla, indexrelname AS indice,
9         idx_scan AS veces_usado
10 FROM pg_stat_user_indexes
11 WHERE schemaname = 'public'
12        AND idx_scan > 0
13 ORDER BY idx_scan DESC;
14
15 -- Actualizar estadísticas del planificador
16 ANALYZE VERBOSE public.simulacion;
17 ANALYZE VERBOSE public.decision;
18 ANALYZE VERBOSE public.historial_acceso;

```

Listing 7.5: Consulta de estado y uso de índices

Capítulo 8

Conclusiones

El proyecto **SBVIA** cumple con todos los criterios de la rúbrica del Examen Final de Administración de Bases de Datos:

1. **Diseño de la BD y millón de registros.** La base de datos PostgreSQL 16 está correctamente estructurada con 21 tablas, relaciones coherentes, 20 restricciones CHECK, 4 triggers de automatización y 5 vistas. El script de carga masiva genera $\approx 1,372,000$ registros distribuidos coherentemente, superando el mínimo de 1.000.000 solicitado.
2. **Implementación y gestión de respaldos.** Se documentó el procedimiento completo con `pg_dump` en formato `custom`, su ejecución desde Docker, y la restauración con `pg_restore`. El archivo `database/current/schema.sql` constituye el respaldo del esquema completo (62,888 bytes, generado con `pg_dump`).
3. **Auditoría de operaciones.** El sistema audita automáticamente todos los accesos mediante la tabla `historial_acceso` y el trigger `trg_actualizar_ultimo_acceso`. Se detectan intentos fallidos consecutivos para prevenir ataques de fuerza bruta.
4. **Control de usuarios, roles y permisos.** Se implementó RBAC con tres roles de aplicación y tres usuarios PostgreSQL con permisos diferenciados. `sbvia_readonly` solo puede leer; `sbvia_app` puede ejecutar DML; `sbvia_admin` tiene control total.
5. **Índices y optimización.** 13 índices B-Tree compuestos. EXPLAIN ANALYZE demuestra mejoras de hasta **546×** en la consulta más frecuente, pasando de 284.5 ms (Sequential Scan) a 0.52 ms (Index Scan sobre 30,000 filas).

Referencias

PostgreSQL 16 The PostgreSQL Global Development Group. (2023). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>

pg_dump PostgreSQL. (2023). *pg_dump documentation*. <https://www.postgresql.org/docs/16/app-pgdump.html>

EXPLAIN PostgreSQL. (2023). *Using EXPLAIN*. <https://www.postgresql.org/docs/16/using-explain.html>

PL/pgSQL PostgreSQL. (2023). *PL/pgSQL Procedural Language*. <https://www.postgresql.org/docs/16/plpgsql.html>

OWASP OWASP Foundation. (2021). *OWASP Top Ten 2021*. <https://owasp.org/www-project-top-ten>

RBAC Ferraiolo, D., & Kuhn, D. R. (1992). *Role-Based Access Controls*. Proceedings of the 15th NIST-NCSC National Computer Security Conference.

Repositorio Cruz Pérez, J. K., Umaginga Arévalo, J. M., & Zamora Bumbila, D. A. (2026). *SBVIA-AppWeb* (v1.0.0) [Software]. <https://github.com/keithdrox/SBVIA-AppWeb>